CS 311 — ALGORITHM DESIGN AND ANALYSIS

Lecture 8 — Question Bank

Minimum Spanning Tree (Kruskal) & Single-Source Shortest Path (Dijkstra)

بنك أسئلة شامل — أسئلة اختيار من متعدد، أسئلة مقالية، ومسائل حسابية — مع الإجابات النموذجية والشروح بالعربي

50	6	12	32
إجمالي الأسئلة	مسألة حسابية	سؤال مقالي	سؤال اختيار من متعدد

القسم الأول: أسئلة الاختيار من متعدد (MCQ)

1 **1. In a weighted adjacency matrix, what value is stored at A[i][j] when there is NO edge between vertex i and vertex j (i != j)?**

الترجمة: في مصفوفة التجاور الموزونة، ما القيمة المخزنة في A[i][j] عندما لا يوجد حافة بين الرأسين i و j (i != j)؟

A. 0

B. w(i, j)

C. Infinity

D. -1

✅ **الإجابة الصحيحة: C — Infinity**

💡 **ليه صح:** الصحيح هو Infinity لأن عدم وجود حافة بين i و j (والفرق بينهما غير صفري) يُمثل بـ infinity. أما 0 فهو للقطر A[i][i] فقط، و w(i,j) هو الوزن لو كانت الحافة موجودة.

📍 **المرجع:** Lecture 8 — Weighted Adjacency Matrix slide | likely: high

Graph Representation | Weighted Adjacency Matrix

2 **2. The Hand-Shaking Lemma for any undirected graph states that:**

الترجمة: تنص Hand-Shaking Lemma لأي رسم بياني غير موجّه على أن:

A. sum of degrees = |V|

B. sum of degrees = 2|E|

C. sum of degrees = |E|

D. sum of degrees = |V| + |E|

✅ **الإجابة الصحيحة: B — sum of degrees = 2|E|**

💡 **ليه صح:** الصحيح هو أن مجموع درجات كل الرؤوس = 2 × عدد الحافات، لأن كل حافة تساهم بدرجتين (تلمس رأسين). لو كان |E| بس يبقى نص الحافة، ولو |V| يبقى مفيش علاقة مباشرة.

📍 **المرجع:** Lecture 8 — The Hand-Shaking Lemma slide | likely: high

Graph Theory | Hand-Shaking Lemma

3 **3. For any directed graph, the sum of in-degrees of all vertices equals:**

الترجمة: لأي رسم بياني موجّه، مجموع in-degrees لكل الرؤوس يساوي:

A. |V|

B. 2|E|

C. |E|

D. |V| - 1

✅ **الإجابة الصحيحة: C — |E|**

💡 **ليه صح:** الصحيح هو |E| لأن كل حافة موجهة لها رأس واحد مصدر وواحد وجهة، فتساهم بـ 1 في الـ out-degree للمصدر و 1 في الـ in-degree للوجهة. وبالتالي مجموع الـ in-degrees = مجموع الـ out-degrees| = |E|. الـ 2|E| هي للـ Hand-Shaking للموجه وغير الموجه.

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma (directed form) | likely: high

Graph Theory | Directed Graph | Hand-Shaking Lemma

4 **4. In Kruskal's algorithm, the edges are sorted in:**

الترجمة: في خوارزمية Kruskal، تُفرز الحافات بترتيب:

A. non-increasing order of weight

B. non-decreasing order of weight

C. lexicographic order of vertex labels

D. random order

✅ **الإجابة الصحيحة: B — non-decreasing order of weight**

💡 **ليه صح:** الصحيح هو non-decreasing (تصاعدي) لأوزان الحافات عشان نختار أخف حافة اولًا في كل خطوة (greedy). non-increasing معناه تنازلي وده غلط، lexico مش بيعتمد على الوزن، random بيبطل ميزة greedy.

📍 **المرجع:** Lecture 8 — Kruskal's Algorithm High-Level Pseudocode | likely: high

Greedy | MST | Kruskal

5 **5. Which data structure does Kruskal's algorithm use to efficiently detect cycles?**

الترجمة: ما بنية البيانات التي تستخدمها خوارزمية Kruskal لكشف الحلقات بكفاءة؟

A. Stack

B. Queue

C. Disjoint Set (Union-Find)

D. Hash Table

✅ **الإجابة الصحيحة: C — Disjoint Set (Union-Find)**

💡 **ليه صح:** الصحيح هو Disjoint Set (Union-Find) لأنها بتتيح MakeSet و FindSet و Union بكفاءة عالية، و FindSet لو الأخرجتين في نفس المجموعة يبقى الحافة هتعمل cycle. Stack/Queue مش بنوكشف cycles، Hash Table أبطأ في عمليات الـ Union.

📍 **المرجع:** Lecture 8 — Kruskal's Algorithm pseudocode (MakeSet/FindSet/Union) likely: high

Union-Find Disjoint Set Kruskal

6 **6. The overall time complexity of Kruskal's algorithm is:**

الترجمة: التعقيد الزمني الكلي لخوارزمية Kruskal هو:

A. O(n^2)

B. O(m log n)

C. O(m log m)

D. O(n log n)

✅ **الإجابة الصحيحة: C — O(m log m)**

💡 **ليه صح:** الصحيح هو O(m log m) حيث |E| = m|. السبب أن الفرز بياخد O(m log m) وده أكبر حد. لو قلنا O(m log n) ده بعد استبعاد الفرز مش صح ككلي، O(n^2) غلط تمامًا. والـ O(n log n) بيتجاهل عدد الحافات.

📍 **المرجع:** Lecture 8 — Kruskal's Algorithm: Running Time likely: high

Time Complexity Kruskal

7 7. In Prim's algorithm, the intermediate solution is a:

الترجمة: في خوارزمية Prim، الحل الوسيط هو:

A. spanning forest

B. subtree

C. single cycle

D. disconnected set of edges

✅ **الإجابة الصحيحة: B — subtree**

💡 **ليه صح:** الصحيح هو subtree لأن Prim بيكبر شجرة واحدة فقط بإضافة أخف حافة تربط رأس في الشجرة برأس برّه. أما spanning forest فده وصف Kruskal، والـ cycle مش هدف MST، و disconnected set مش ممكن لإن Prim بيبني connected.

📍 **المرجع:** Lecture 8 — Finding a MST (Prim vs Kruskal) | likely: high

Comparison MST Prim

8 8. In Kruskal's algorithm, the intermediate solution is a:

الترجمة: في خوارزمية Kruskal، الحل الوسيط هو:

A. subtree

B. spanning forest

C. single cycle

D. complete graph

✅ **الإجابة الصحيحة: B — spanning forest**

💡 **ليه صح:** الصحيح هو spanning forest لأن Kruskal بيضيف الحافات الأرخص بغض النظر عن الترابط، فيمكن تكون مكونات منفصلة (forest) لحد ما تتحد. subtree هو وصف Prim، cycle غلط لأننا بنمنع cycles، و complete graph غير منطقي.

📍 **المرجع:** Lecture 8 — Finding a MST (Prim vs Kruskal) | likely: high

Comparison MST Kruskal

9 **9. What problem does Dijkstra's algorithm solve?**

الترجمة: ما المشكلة التي تحلها خوارزمية Dijkstra؟

A. Minimum Spanning Tree

B. Single-Source Shortest Paths

C. All-Pairs Shortest Paths

D. Maximum Flow

✅ **الإجابة الصحيحة: B — Single-Source Shortest Paths**

💡 **ليه صح:** الصحيح هو Single-Source Shortest Paths أي أقصر مسار من مصدر واحد إلى كل باقي الرؤوس. MST بتحله خوارزميات تانية (Prim/Kruskal)، All-Pairs بيستخدم Floyd-Warshall، Maximum Flow بيستخدم Ford-Fulkerson.

📍 **المرجع:** Lecture 8 — Single-Source Shortest-Paths Problem / Dijkstra's Algorithm | likely: high

Shortest Path Dijkstra

10 **10. Dijkstra's algorithm requires that all edge weights be:**

الترجمة: تتطلب خوارزمية Dijkstra أن تكون أوزان الحافات:

A. positive only

B. non-negative

C. negative

D. any real number

✅ **الإجابة الصحيحة: B — non-negative**

💡 **ليه صح:** الصحيح هو non-negative (غير سالبة، يعني تسمح بصفر) لأن Dijkstra بتفترض إن لما رأس يتحدد، المسار الأقصر ليه خلص، وهذا الافتراض بيبطل لو في أوزان سالبة. positive only يرفض الأصفار ومش لازم، negative بيفسد الخوارزمية.

📍 **المرجع:** Lecture 8 — Single-Source Shortest-Paths Problem statement | likely: high

Constraints Dijkstra

11 **11. The time complexity of Dijkstra's algorithm using a binary heap is:**

الترجمة: التعقيد الزمني لخوارزمية Dijkstra باستخدام binary heap هو:

A. O(V^2)

B. O((V + E) log V)

C. O(V * E)

D. O(E log E)

✅ **الإجابة الصحيحة: B — O((V + E) log V)**

💡 **ليه صح:** الصحيح هو O((V + E) log V). V iteration لكل DeleteMin بـ log V، وكل حافة ممكن تعمل UpHeap بـ log V. O(V^2) للنسخة الـ simple بدون heap، O(V*E) Bellman-Ford، O(E log E) مش معتاد لـ Dijkstra.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode and Big-O note | likely: high

Time Complexity Dijkstra

12 **12. In the Dijkstra pseudocode, what is the initial value of dist[s] for the source vertex s?**

الترجمة: في pseudocode الخاص بـ Dijkstra، ما القيمة الابتدائية لـ dist[s] للرأس المصدر s؟

A. infinity

B. 0

C. -1

D. the largest edge weight

✅ **الإجابة الصحيحة: B — 0**

💡 **ليه صح:** الصحيح هو 0 لأن المسافة من المصدر لنفسه = 0. infinity لباقي الرؤوس، -1 مش قيمة منطقية، أكبر وزن حافة غلط لإنها لازم تبدأ من 0 للمصدر.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode lines 1-2 | likely: high

Initialization Dijkstra

13 **13. What is the role of the predecessor pointer p[v] in Dijkstra's algorithm?**

الترجمة: ما دور المؤشر السابق p[v] في خوارزمية Dijkstra؟

A. stores the distance

B. stores the previous vertex on the shortest path

C. stores the edge weight

D. stores the priority value

✅ **الإجابة الصحيحة: B — stores the previous vertex on the shortest path**

💡 **ليه صح:** الصحيح هو storing previous vertex على المسار الأقصر ل reconstruct المسار. dist[] بيخزن المسافة، w(u,v) هو الوزن، priority بيخزنه الـ heap داخليًا.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode line 9 | likely: high

Path Reconstruction Dijkstra

14 **14. In a weighted adjacency list, what does Adj[i] contain for each vertex i?**

الترجمة: في قائمة التجاور الموزونة، ماذا تحتوي Adj[i] لكل رأس i؟

A. only the neighbor vertex ids

B. pairs (j, w(i, j)) for each neighbor j

C. only the edge weights

D. the degree of vertex i only

✅ **الإجابة الصحيحة: B — pairs (j, w(i, j)) for each neighbor j**

💡 **ليه صح:** الصحيح هو pairs (j, w(i,j)) أي كل جار مع وزن الحافة ليه. لو بس ids يبقى list عادية مش موزونة، أوزان بس بتفقد معلومات الجيران، و degree بس مفيهاش التفاصيل.

📍 **المرجع:** Lecture 8 — (Weighted) Adjacency List Structure | likely: high

Graph Representation Adjacency List

15 **15. For a directed graph, the degree of vertex v equals:**

الترجمة: للرسم البياني الموجه، درجة الرأس v تساوي:

A. outdeg(v) only

B. indeg(v) only

C. outdeg(v) + indeg(v)

D. outdeg(v) * indeg(v)

✅ **الإجابة الصحيحة: C — outdeg(v) + indeg(v)**

💡 **ليه صح:** الصحيح هو outdeg + indeg لأن كل حافة موجهة إما خارجة من v أو داخلة ليه. لو حد فقط يبقى نص المعلومة، وضرب غلط تمامًا.

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma slide | likely: high

Hand-Shaking | Degree | Directed Graph

16 **16. Which of the following is NOT a typical MST application mentioned in the lecture?**

الترجمة: أيٌّ مما يلي ليس من تطبيقات MST المذكورة في المحاضرة؟

A. telephone network design

B. electrical power network

C. sorting an array of numbers

D. road network design

✅ **الإجابة الصحيحة: C — sorting an array of numbers**

💡 **ليه صح:** الصحيح هو sorting an array لأن ده مش مرتبط بـ MST. المحاضرة ذكرت telephone, electrical power, hydraulic, TV cable, computer, road networks.

📍 **المرجع:** Lecture 8 — MST Applications slide | likely: high

Network Design | MST Applications

17 **17. In Dijkstra's algorithm, after a vertex is selected (DeleteMin) and processed, its shortest distance is:**

الترجمة: في خوارزمية Dijkstra، بعد اختيار رأس (DeleteMin) ومعالجته، مسافته الأقصر:

A. still subject to update

B. finalized and never changes

C. set to infinity

D. set to -1

✅ **الإجابة الصحيحة: B — finalized and never changes**

💡 **ليه صح:** الصحيح هو finalized وده اللي بيخلي الخوارزمية greedy. لو مع weights غير سالبة، أول ما نطلع رأس من الـ Q يبقى مسافته مضايقتش. تحديث infinity أو -1 غلط.

📍 **المرجع:** Lecture 8 — Dijkstra's Algorithm description | likely: high

Greedy Property | Dijkstra

18 **18. The number of edges in a Minimum Spanning Tree of a connected graph with n vertices is:**

الترجمة: عدد الحافات في MST لرسم بياني متصل بـ n رؤوس هو:

A. n

B. n - 1

C. n + 1

D. 2n - 1

✅ **الإجابة الصحيحة: B — n - 1**

💡 **ليه صح:** الصحيح هو n-1 لأن أي spanning tree لرسم متصل بـ n رؤوس لازم يكون فيه n-1 حافة بالظبط (لا دورايز ولا غير متصل).

📍 **المرجع:** Lecture 8 — Kruskal high-level pseudocode (while encounter < |V|-1) | likely: high

Spanning Tree Properties | MST

19 **19. The MakeSet(v) operation in Kruskal's algorithm is used to:**

الترجمة: عملية MakeSet(v) في خوارزمية Kruskal تُستخدم لـ:

A. sort edges

B. create a singleton set containing only v

C. merge two sets

D. find the minimum edge

✅ **الإجابة الصحيحة: B — create a singleton set containing only v**

💡 **ليه صح:** الصحيح هو create singleton set for v أي مجموعة فيها v بس. sort بتتم بمعزل، merge هي Union، find minimum بيتم بترتيب الفرز.

📍 **المرجع:** Lecture 8 — Kruskal pseudocode (for each v MakeSet(v)) | likely: high

MakeSet | Disjoint Set | Kruskal

20 **20. The condition to add an edge (u, v) in Kruskal's algorithm is:**

الترجمة: شرط إضافة حافة (u, v) في خوارزمية Kruskal هو:

A. FindSet(u) == FindSet(v)

B. FindSet(u) != FindSet(v)

C. w(u,v) is minimum

D. u and v are both in tree

✅ **الإجابة الصحيحة: B — FindSet(u) != FindSet(v)**

💡 **ليه صح:** الصحيح هو FindSet(u) != FindSet(v) يعني u و v في مجموعتين مختلفتين فالحافة مش هتعمل cycle. لو نفس المجموعة يبقى cycle ونمنعها. الوزن الأقل مش كفاية لو هيعمل cycle.

📍 **المرجع:** Lecture 8 — Kruskal pseudocode (if FindSet(u) != FindSet(v)) | likely: high

Union-Find | Cycle Detection | Kruskal

21 21. How many FindSet() calls are made during Kruskal's algorithm?

الترجمة: كم عدد استدعاءات FindSet() أثناء خوارزمية Kruskal؟

A. O(n)

B. O(m)

C. O(n log n)

D. O(m^2)

✅ **الإجابة الصحيحة: B — O(m)**

💡 **ليه صح:** الصحيح هو O(m) لأن لكل حافة في E بنستدعي FindSet مرتين (مرة لـ u ومرة لـ v)، فالمجموع 2m = O(m). O(n) لـ MakeSet و Union بس.

📍 **المرجع:** Lecture 8 — Kruskal Running Time slide (O(m) FindSet calls) | likely: high

Complexity Analysis | Kruskal

22 22. In Dijkstra's algorithm, the priority queue stores vertices ordered by:

الترجمة: في خوارزمية Dijkstra، طابور الأولويات يخزن الرؤوس مرتبين بـ:

A. vertex id

B. their current dist[] value

C. edge count

D. alphabetical order

✅ **الإجابة الصحيحة: B — their current dist[] value**

💡 **ليه صح:** الصحيح هو current dist[] value أي أقل مسافة معروفة لكل رأس. ده اللي بيخلّي DeleteMin يرجع أقرب رأس غير معالج. vertex id / alphabet / edge count مش ليها معنى في Dijkstra.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode (ConstructMinHeap(V, dist)) | likely: high

Priority Queue | Dijkstra

23 **23. The Hand-Shaking Lemma implies that in any undirected graph, the number of vertices with odd degree is:**

الترجمة: تستلزم Hand-Shaking Lemma أنه في أي رسم غير موجّه، عدد الرؤوس ذات الدرجة الفردية:

A. always odd

B. always even

C. always zero

D. always equal to |V|

✅ **الإجابة الصحيحة: B — always even**

💡 **ليه صح:** الصحيح هو always even لأن لو جمع الدرجات = 2|E| (زوجي)، فلازم مجموع الدرجات الفردية يكون زوجي، فعدد الرؤوس ذات الدرجة الفردية لازم يكون زوجي. odd مستحيل، صفر ممكن بس مش دائمًا.

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma + standard corollary | likely: high

Graph Theory | Hand-Shaking Lemma

24 **24. The Dijkstra pseudocode uses which heap operation to update a vertex's distance after relaxation?**

الترجمة: pseudocode الخاص بـ Dijkstra يستخدم عملية heap ما لتحديث مسافة رأس بعد relaxation؟

A. DeleteMin

B. UpHeap

C. Heapify

D. BuildHeap

✅ **الإجابة الصحيحة: B — UpHeap**

💡 **ليه صح:** الصحيح هو UpHeap لأنها بترفع الرأس لو قلّت dist بتاعته عشان يحافظ على خاصية min-heap. DeleteMin لإخراج أصغر عنصر، Heapify/BuildHeap لإنشاء الـ heap مرة واحدة.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode line 10 (UpHeap(v,Q)) | likely: high

Priority Queue | Heap | Dijkstra

25 **25. The greedy strategy common to both Prim and Dijkstra is:**

الترجمة: الاستراتيجية الجشعة المشتركة بين Prim و Dijkstra هي:

A. always sort all edges first

B. repeatedly pick the locally best (minimum) option

C. use recursion

D. use dynamic programming

✅ **الإجابة الصحيحة: B — repeatedly pick the locally best (minimum) option**

💡 **ليه صح:** الصحيح هو repeatedly pick locally best أي في كل خطوة تختار العنصر الأقل حاليًا.
الفرز الشامل خاص بـ Kruskal، recursion/DP مش مستخدمة في أي منهم.

📍 **المرجع:** Lecture 8 — Prim and Dijkstra both use priority queue greedy selection | likely: high

Comparison Dijkstra Prim Greedy

26 **26. Which operation is performed inside the inner for-loop of Dijkstra's algorithm?**

الترجمة: ما العملية التي تتم داخل for-loop الداخلية في خوارزمية Dijkstra؟

A. BuildHeap

B. Relaxation (if dist[v] > dist[u] + w(u,v) then update)

C. MakeSet

D. Sort edges

✅ **الإجابة الصحيحة: B — Relaxation (if dist[v] > dist[u] + w(u,v) then update)**

💡 **ليه صح:** الصحيح هو relaxation (الشرط if dist[v] > dist[u] + w(u,v)) وهي قلب الخوارزمية.
BuildHeap يتم مرة واحدة في البداية، MakeSet و Sort خاصة بـ Kruskal.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode lines 7-10 | likely: high

Relaxation Dijkstra

27 **27. In Kruskal's algorithm, what is the role of the Union operation?**

الترجمة: في خوارزمية Kruskal، ما دور عملية Union؟

A. sort edges

B. merge the sets containing u and v after adding edge (u,v)

C. find the smallest edge

D. initialize the tree

✅ **الإجابة الصحيحة: B — merge the sets containing u and v after adding edge (u,v)**

💡 **ليه صح:** الصحيح هو merge sets containing u and v لما نضيف الحافة لTree عشان لو في حاجة تانية بعدها حاول تربط u بـ v ب FindSet نلاقيهم نفس المجموعة. sort تم قبل، find minimum بترتيب الفرز، initialize بـ MakeSet.

📍 **المرجع:** Lecture 8 — Kruskal pseudocode (Union(FindSet(u), FindSet(v))) | likely: high

Disjoint Set | Union-Find | Kruskal

28 **28. For an undirected graph G, the degree of vertex v equals:**

الترجمة: للرسم البياني غير الموجه G، درجة الرأس v تساوي:

A. |Adj[v]|

B. 2 * |Adj[v]|

C. |Adj[v]| / 2

D. |V| - |Adj[v]|

✅ **الإجابة الصحيحة: A — |Adj[v]|**

💡 **ليه صح:** الصحيح هو |Adj[v]| لإن Adj[v] بتحتوي على كل الجيران، وكل جار = حافة واحدة (غير الموجه بتعد مرة). ضرب في 2 غلط لأنها بتضاعف، القسمة على 2 غلط.

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma slide (deg(v) = |Adj[v]|) | likely: high

Adjacency List | Degree | Graph Theory

29 **29. The ConstructMinHeap step in Dijkstra's pseudocode has time complexity:**

الترجمة: خطوة ConstructMinHeap في pseudocode الخاص بـ Dijkstra لها تعقيد زمني:

A. O(V)

B. O(V log V)

C. O(V^2)

D. O(E)

✅ **الإجابة الصحيحة: B — O(V log V)**

💡 **ليه صح:** الصحيح هو O(V log V) لإدراج عناصر V في min-heap. ممكن يكون O(V) لو BuildHeap لكن الكلاسيكي O(V log V). O(V^2) كبير أوي، O(E) مش صحيح لإن الموضوع رؤوس.

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode line 3 | likely: medium

Time Complexity | Heap Construction | Dijkstra

30 **30. Which algorithm does NOT use the Disjoint Set (Union-Find) data structure?**

الترجمة: أي خوارزمية لا تستخدم بنية Disjoint Set (Union-Find)؟

A. Kruskal's MST algorithm

B. Dijkstra's shortest path algorithm

C. Both use it

D. Neither uses it

✅ **الإجابة الصحيحة: B — Dijkstra's shortest path algorithm**

💡 **ليه صح:** الصحيح هو Dijkstra لإنها بتستخدم priority heap فقط. Kruskal هو اللي بيستخدم Union-Find لكشف cycles.

📍 **المرجع:** Lecture 8 — Kruskal pseudocode + Dijkstra pseudocode | likely: high

Comparison | Disjoint Set | Dijkstra | Kruskal

31 **31. The condition to terminate Kruskal's while-loop is when:**

الترجمة: شرط إنهاء حلقة while في خوارزمية Kruskal هو عندما:

A. all edges are processed

B. encounter == |V| - 1

C. encounter == |V|

D. no more edges remain

✅ **الإجابة الصحيحة: B — encounter == |V| - 1**

💡 **ليه صح:** الصحيح هو encounter == |V| - 1 لأن MST لـ |V| رؤوس يحتاج |V|-1 حافة بالظبط. |V| غلط لإنه يعمل all edges processed بطيء وغير ضروري بعد ما نوصل لـ |V|-1. cycle.

📍 **المرجع:** Lecture 8 — Kruskal high-level pseudocode (while encounter < |V|-1) | likely: high

Termination Condition | Kruskal

32 **32. Which statement correctly compares Prim and Kruskal?**

الترجمة: أي جملة تقارن بشكل صحيح بين Prim و Kruskal؟

A. Prim grows a forest; Kruskal grows a single tree

B. Prim grows a single tree; Kruskal grows a forest

C. Both grow a single tree

D. Both grow a forest

✅ **الإجابة الصحيحة: B — Prim grows a single tree; Kruskal grows a forest**

💡 **ليه صح:** الصحيح هو Prim grows single tree, Kruskal grows forest. ده الفرق الأساسي المذكور في المحاضرة. العكس خلط، و both واحد غلط.

📍 **المرجع:** Lecture 8 — Finding a MST slide | likely: high

MST | Comparison | Kruskal | Prim

القسم الثاني: الأسئلة المقالية (Essay Questions)

E1 **1. Define a Minimum Spanning Tree (MST) and list three of its real-world applications mentioned in the lecture.**

الترجمة: عرّف Minimum Spanning Tree (MST) واذكر ثلاثة تطبيقات واقعية له مذكورة في المحاضرة.

✅ **الإجابة النموذجية:**

A Minimum Spanning Tree (MST) of a connected, weighted, undirected graph G = (V, E) is a subset of edges that:
1. Connects all the vertices together (i.e., it is a spanning subgraph),
2. Contains no cycles (i.e., it is a tree),
3. Has the minimum possible total edge weight among all such spanning trees.
A spanning tree of a graph with n vertices always contains exactly (n - 1) edges.
Applications mentioned in the lecture:
- Network design: telephone networks, electrical power networks, hydraulic networks, TV cable networks, computer networks, and road networks.
- MST gives the minimum-cost network connecting all sites.
- MST is the most economical construction of a network.

📍 **المرجع:** Lecture 8 — Finding a MST + MST Applications slides | likely: high

Applications Definition MST

E2 **2. Compare Prim's and Kruskal's algorithms for finding MST in terms of the intermediate solution, the strategy used, and the data structures required.**

الترجمة: قارن بين خوارزمية Prim و Kruskal لإيجاد MST من حيث: الحل الوسيط، الاستراتيجية، وبنى البيانات المطلوبة.

✅ **الإجابة النموذجية:**

```
Both algorithms are greedy and find an MST, but they differ in strategy and
intermediate state:
1. Intermediate solution:
   - Prim: the intermediate solution is a single subtree that grows one vertex
at a time.
   - Kruskal: the intermediate solution is a spanning forest (possibly
disconnected components) that gradually unifies.
2. Strategy:
   - Prim: repeatedly adds the least-cost edge that connects a vertex in the
existing tree to a vertex NOT in the existing tree.
   - Kruskal: repeatedly adds the least-cost edge that does NOT introduce a
cycle among the edges included so far.
3. Data structures:
   - Prim: a priority queue (min-heap) keyed by edge weights connecting tree to
non-tree vertices.
   - Kruskal: a Disjoint Set (Union-Find) data structure to detect cycles
efficiently via MakeSet, FindSet, and Union operations.
4. Time complexity:
   - Prim: O((n + m) log n) using a binary heap.
   - Kruskal: O(m log m) dominated by the sorting step.
```

📍 **المرجع:** Lecture 8 — Finding a MST + Kruskal pseudocode + Dijkstra note | likely: high

MST Comparison Kruskal Prim

E3 **3. Explain Kruskal's algorithm step by step using the pseudocode shown in the lecture.**

الترجمة: اشرح خوارزمية Kruskal خطوة بخطوة باستخدام pseudocode المعروض في المحاضرة.

✅ **الإجابة النموذجية:**

```
Kruskal's algorithm (greedy MST construction):
Step 1: Initialize T as an empty set of edges (the future MST).
Step 2: For each vertex v in V, call MakeSet(v) so each vertex becomes its own
singleton set in a Disjoint Set (Union-Find) structure.
Step 3: Sort all edges in E in non-decreasing order of their weight w.
Step 4: Iterate over edges (u, v) in sorted order. For each edge:
   - If FindSet(u) != FindSet(v), then u and v are in different components (no
cycle will be formed):
       * Add edge (u, v) to T.
       * Call Union(FindSet(u), FindSet(v)) to merge the two components.
   - Otherwise, discard the edge (it would form a cycle).
Step 5: Stop when T contains exactly |V| - 1 edges. Return T as the MST.
The algorithm grows a spanning forest that gradually unifies into a single
tree.
```

📍 **المرجع:** Lecture 8 — Kruskal's Algorithm pseudocode | likely: high

Pseudocode | Algorithm Steps | Kruskal

E4 **4. Derive the time complexity of Kruskal's algorithm in terms of n = |V| and m = |E|.**

الترجمة: اشتق التعقيد الزمني لخوارزمية Kruskal بدلالة |V| = n و |E| = m.

✅ **الإجابة النموذجية:**

```
Let n = |V| and m = |E|.
1. MakeSet: O(n) calls, each O(1) amortized in disjoint set with path
compression => total O(n).
2. Sort E by weight: O(m log m) using comparison-based sort.
3. FindSet: O(m) calls (two per edge). With union-by-rank and path compression,
each FindSet is nearly O(1) amortized (inverse Ackermann), but a tighter bound
yields O(m log n) or essentially O(m alpha(n)).
4. Union: at most O(n) calls, each O(1) amortized.
5. Dominant term: O(m log m) for sorting edges.
Since m <= n(n-1)/2, we have log m <= 2 log n, so O(m log m) is essentially O(m
log n) for connected graphs.
Overall complexity: O(m log m) (equivalently O(m log n) for connected graphs).
```

likely: high | 📍 **المرجع:** Lecture 8 — Kruskal's Algorithm: Running Time slide

Analysis | Time Complexity | Kruskal

E5 **5. State the Hand-Shaking Lemma for both undirected and directed graphs, and explain why it holds.**

الترجمة: اذكر Hand-Shaking Lemma للرسوم البيانية الموجهة وغير الموجهة، واشرح لماذا تنطبق.

✅ **الإجابة النموذجية:**

```
Undirected form: For any undirected graph G = (V, E),
   sum over v in V of deg(v) = 2 * |E|.
Directed form: For any directed graph G = (V, E),
   sum over v in V of in-deg(v) = sum over v in V of out-deg(v) = |E|.
   Also, deg(v) = out-deg(v) + in-deg(v).
Explanation: Each undirected edge is incident to exactly two vertices, so it
contributes 1 to the degree of each of its endpoints, hence 2 to the sum. Each
directed edge has exactly one source (contributing 1 to out-deg) and one
destination (contributing 1 to in-deg), so the total in-degree equals the total
out-degree equals the number of edges.
Corollary: In any undirected graph, the number of vertices with odd degree is
even.
```

likely: high | 📍 **المرجع:** Lecture 8 — The Hand-Shaking Lemma slide

Proof | Graph Theory | Hand-Shaking Lemma

E6 **6. Explain Dijkstra's algorithm step by step using the pseudocode shown in the lecture.**

الترجمة: اشرح خوارزمية Dijkstra خطوة بخطوة باستخدام pseudocode المعروض في المحاضرة.

✅ **الإجابة النموذجية:**

```
Dijkstra's algorithm (single-source shortest paths with non-negative weights):
Step 1: For each vertex u in V(G), set dist[u] = infinity.
Step 2: Set dist[s] = 0 and p[s] = nil (s is the source; it will be the first
vertex scanned).
Step 3: Build a min-heap Q containing all vertices, keyed by their current
dist[] value (ConstructMinHeap).
Step 4: While Q is not empty (|V| iterations):
   - u = DeleteMin(Q) -> removes the vertex with the smallest dist[] (O(log V)
per deletion).
   - For each neighbor v in Adj[u]:
      * If dist[v] > dist[u] + w(u, v) then:
          - Update dist[v] = dist[u] + w(u, v).
          - Set p[v] = u (record predecessor for path reconstruction).
          - UpHeap(v, Q) to restore the heap property.
Step 5: Stop when Q is empty. The dist[] array now holds shortest distances
from s.
Total time: O(E log V) using a standard binary heap.
```

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode | likely: high

Pseudocode | Algorithm Steps | Dijkstra

E7 7. Derive the time complexity of Dijkstra's algorithm using a binary heap.

الترجمة: اشتق التعقيد الزمني لخوارزمية Dijkstra باستخدام binary heap.

✅ الإجابة النموذجية:

```
Let n = |V| and m = |E|.
1. Initialize dist[]: O(n).
2. ConstructMinHeap: O(n log n) using repeated insertion (or O(n) with
BuildHeap).
3. The while loop iterates n times (once per vertex). Each iteration:
   - DeleteMin: O(log n).
   - Inner for-loop scans all neighbors of u. Across all iterations, each edge
is examined at most twice (once from each endpoint), so total O(m) relaxations.
   - Each successful relaxation calls UpHeap: O(log n).
4. Therefore:
   - n DeleteMin operations: O(n log n).
   - O(m) UpHeap operations (worst case): O(m log n).
5. Total time = O(n log n) + O(m log n) = O((n + m) log n) = O(E log V) using
the lecture's notation.
Note: A Fibonacci heap can improve the bound to O(m + n log n), but the lecture
focuses on the binary heap version.
```

📍 **المرجع:** Lecture 8 — Dijkstra pseudocode + Big-O note | likely: high

Analysis | Time Complexity | Dijkstra

E8 8. Define the Single-Source Shortest-Paths problem and mention one real-world application.

الترجمة: عرّف مشكلة Single-Source Shortest-Paths واذكر تطبيقًا واقعيًا واحدًا لها.

✅ الإجابة النموذجية:

```
Problem statement: Given a connected graph G with non-negative weights on the
edges and a source vertex s in G, determine the shortest paths from s to all
other vertices in G.
Formally: compute, for every vertex v in V(G), the minimum total weight of any
path from s to v.
Real-world application: road map applications (e.g., GPS navigation), where the
graph models a road network, vertices are intersections, edge weights are
travel times or distances, and the user wants the shortest route from their
current location to every other destination. Other applications include network
routing protocols such as OSPF and game AI pathfinding.
```

📍 **المرجع:** Lecture 8 — Single-Source Shortest-Paths Problem slide | likely: high

Applications | Problem Definition | Shortest Path

E9 **9. Explain the role of the Disjoint Set (Union-Find) data structure in Kruskal's algorithm. Mention its three core operations and what each does.**

الترجمة: اشرح دور بنية Disjoint Set (Union-Find) في خوارزمية Kruskal. اذكر العمليات الأساسية الثلاث وماذا تفعل كل واحدة.

✅ **الإجابة النموذجية:**

```
Role: The Disjoint Set (Union-Find) data structure allows Kruskal's algorithm
to efficiently test whether adding an edge would create a cycle, and to merge
components when an edge is added.
Three core operations:
1. MakeSet(v): creates a new singleton set containing only vertex v. Called
once for each vertex during initialization (O(n) calls).
2. FindSet(v): returns the representative (root) of the set that contains
vertex v. Two vertices are in the same connected component iff their FindSet
results are equal.
3. Union(u, v): merges the two sets that contain u and v into a single set.
Called after adding a safe edge to reflect that the two endpoints are now
connected.
In Kruskal's main loop, an edge (u, v) is added to T only if FindSet(u) !=
FindSet(v) (different components => no cycle). After adding it, Union(u, v) is
called to merge the components.
Using path compression and union by rank, all three operations are nearly
constant (inverse Ackermann) amortized, which keeps Kruskal's overall
complexity at O(m log m).
```

📍 **المرجع:** Lecture 8 — Kruskal pseudocode (MakeSet/FindSet/Union) | likely: high

Data Structures · Union-Find · Disjoint Set · Kruskal

E10 10. Compare weighted adjacency matrix and weighted adjacency list representations in terms of space and use cases.

الترجمة: قارن بين تمثيل مصفوفة التجاور الموزونة وقائمة التجاور الموزونة من حيث المساحة وحالات الاستخدام.

✅ الإجابة النموذجية:

```
Weighted adjacency matrix:
- An n x n matrix A where A[i][j] = w(i, j) if (i, j) is in E, A[i][i] = 0, and
A[i][j] = infinity otherwise (no edge).
- Space: O(n^2), regardless of how many edges the graph has.
- Best suited for dense graphs where m is close to n^2, or when fast O(1) edge-
weight lookup is required.
- Iterating over neighbors of v requires scanning an entire row of n entries.
Weighted adjacency list:
- An array Adj[1..n] where Adj[i] is a list of pairs (j, w(i, j)) for every
neighbor j of vertex i.
- Space: O(n + m), proportional to the actual number of edges.
- Best suited for sparse graphs where m is much smaller than n^2 (most real-
world graphs).
- Iterating over neighbors of v takes O(deg(v)) time, which is exactly what
graph traversal algorithms need.
Most MST and shortest-path algorithms (Prim, Kruskal, Dijkstra) prefer the
adjacency list representation for sparse graphs.
```

📍 **المرجع:** Lecture 8 — Weighted Adjacency Matrix + (Weighted) Adjacency List Structure slides |
likely: high

Graph Representation | Adjacency List | Adjacency Matrix

E11 **11. Why does Dijkstra's algorithm NOT work with negative edge weights? Give a brief conceptual justification.**

الترجمة: لماذا لا تعمل خوارزمية Dijkstra مع الأوزان السالبة؟ اذكر تبريرًا مفاهيميًا موجزًا.

✅ **الإجابة النموذجية:**

Dijkstra's algorithm relies on a greedy invariant: once a vertex u is extracted from the priority queue via DeleteMin, its dist[u] is considered finalized (the shortest possible) and is never updated again.
This invariant holds only when all edge weights are non-negative. With non-negative weights, any future path that goes through other vertices and arrives at u cannot be shorter than the current dist[u], because adding non-negative numbers can only keep the path length the same or increase it.
If even one edge has a negative weight, a later path through a vertex not yet processed might reduce dist[u] after u has been finalized, which violates the invariant. The algorithm would therefore return incorrect (too large) distances.
For graphs with negative weights, the Bellman-Ford algorithm should be used instead.

📍 **المرجع:** Lecture 8 — Single-Source Shortest-Paths Problem statement (non-negative weights)

likely: medium

Limitations | Negative Weights | Dijkstra

E12 **12. List the four standard categories of MST applications mentioned in the lecture and give a one-line example for each.**

الترجمة: اذكر الفئات الأربعة لتطبيقات MST المذكورة في المحاضرة وأعطِ مثالًا من سطر واحد لكل منها.

✅ **الإجابة النموذجية:**

```
The lecture lists MST applications under the general umbrella of network
design, with the following examples:
1. Telephone networks: laying cables to connect a set of telephone exchanges at
minimum total cost.
2. Electrical power networks: connecting power stations and substations with
the minimum total length of transmission lines.
3. Hydraulic (water) networks: designing a pipeline system that links water
sources to consumers at minimum cost.
4. Road networks: building the minimum-cost set of roads that connects a set of
cities so that every city is reachable.
In every case, the MST provides the most economical construction of a network
connecting all sites (vertices) using the minimum total edge weight.
```

📍 **المرجع:** Lecture 8 — MST Applications slide | likely: medium

Network Design | Applications | MST

القسم الثالث: المسائل الحسابية (Calculation Problems)

C1 **1. Verify the Hand-Shaking Lemma on the following undirected graph: V = {A, B, C, D}, E = {(A,B), (A,C), (B,C), (B,D), (C,D)}. Show all degrees and confirm the sum equals 2|E|.**

الترجمة: تحقق من Hand-Shaking Lemma على الرسم البياني غير الموجه التالي: V = {A, B, C, D}، E = {(A,B), (A,C), (B,C), (B,D), (C,D)}. اعرض كل الدرجات وأكد أن المجموع يساوي 2|E|.

✅ **الحل النموذجي:**

```
Given: |V| = 4 and |E| = 5 (edges: AB, AC, BC, BD, CD).
Compute the degree of each vertex:
  deg(A) = 2 (edges AB, AC)
  deg(B) = 3 (edges AB, BC, BD)
  deg(C) = 3 (edges AC, BC, CD)
  deg(D) = 2 (edges BD, CD)
Sum of degrees = 2 + 3 + 3 + 2 = 10.
By the Hand-Shaking Lemma: sum of degrees = 2 * |E| = 2 * 5 = 10.
Both sides match (10 = 10), so the lemma is verified.
Bonus observation: the number of vertices with odd degree is 2 (B and C), which
is even, consistent with the corollary of the lemma.
```

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma slide | likely: high

Verification | Calculation | Hand-Shaking Lemma

C2 **2. Apply Kruskal's algorithm on the following weighted undirected graph and find the MST. Vertices = {A, B, C, D, E, F}. Edges with weights: (A,B,4), (A,C,3), (B,C,1), (B,D,2), (C,D,4), (C,E,5), (D,E,7), (D,F,6), (E,F,8). Show each step and the total MST weight.**

الترجمة: طبّق خوارزمية Kruskal على الرسم البياني الموزون غير الموجه التالي وأوجد MST. الرؤوس = {A, B, C, D, E, F}. الحافات بأوزانها: (A,B,4), (A,C,3), (B,C,1), (B,D,2), (C,D,4), (C,E,5), (D,E,7), (D,F,6), (E,F,8). اعرض كل خطوة والوزن الكلي لـ MST.

✅ **الحل النموذجي:**

```
Step 1 - Sort edges in non-decreasing order of weight:
  (B,C,1), (B,D,2), (A,C,3), (A,B,4), (C,D,4), (C,E,5), (D,F,6), (D,E,7),
(E,F,8).
Step 2 - Initialize: T = empty, MakeSet for every vertex (each vertex is its
own set). |V| - 1 = 5 edges needed.
Step 3 - Process edges in sorted order:
  - Edge (B,C,1): FindSet(B) != FindSet(C) => no cycle. Add to T. Union(B,C). T
= {(B,C)}. edges = 1.
  - Edge (B,D,2): FindSet(B) != FindSet(D) => no cycle. Add to T. Union(B,D). T
= {(B,C), (B,D)}. edges = 2.
  - Edge (A,C,3): FindSet(A) != FindSet(C) => no cycle. Add to T. Union(A,C). T
= {(B,C), (B,D), (A,C)}. edges = 3.
  - Edge (A,B,4): FindSet(A) == FindSet(B) (both in same set after unions) =>
would create a cycle. SKIP.
  - Edge (C,D,4): FindSet(C) == FindSet(D) => cycle. SKIP.
  - Edge (C,E,5): FindSet(C) != FindSet(E) => no cycle. Add to T. Union(C,E). T
= {(B,C), (B,D), (A,C), (C,E)}. edges = 4.
  - Edge (D,F,6): FindSet(D) != FindSet(F) => no cycle. Add to T. Union(D,F). T
= {(B,C), (B,D), (A,C), (C,E), (D,F)}. edges = 5.
Step 4 - Stop: T now has |V| - 1 = 5 edges. MST is complete.
Final MST edges: (B,C), (B,D), (A,C), (C,E), (D,F).
Total MST weight = 1 + 2 + 3 + 5 + 6 = 17.
```

📍 **المرجع:** Lecture 8 — Kruskal's Algorithm + worked example | likely: high

Step-by-step | Calculation | MST | Kruskal

C3 **3. Apply Dijkstra's algorithm to the following weighted graph and find the shortest distances from source A. Vertices = {A, B, C, D, E}. Edges (undirected, weighted): (A,B,10), (A,C,3), (A,D,20), (B,D,5), (C,B,4), (C,D,11), (D,E,1). Show the dist[] table after each iteration.**

الترجمة: طبّق خوارزمية Dijkstra على الرسم البياني الموزون التالي وأوجد أقصر المسافات من المصدر A. الرؤوس = {A, B, C, D, E}. الحافات (غير موجّهة، موزونة): (A,B,10), (A,C,3), (A,D,20), (B,D,5), (C,B,4), (C,D,11), (D,E,1). اعرض جدول dist[] بعد كل تكرار.

✅ **الحل النموذجي:**

```
Initialization: dist[A]=0, dist[B..E]=infinity. Q = {A,B,C,D,E}.
Iteration 1 - Select A (dist 0):
  Relax A->B: dist[B] = min(inf, 0+10) = 10
  Relax A->C: dist[C] = min(inf, 0+3) = 3
  Relax A->D: dist[D] = min(inf, 0+20) = 20
  dist = {A:0, B:10, C:3, D:20, E:inf}
Iteration 2 - Select C (dist 3, smallest in Q):
  Relax C->B: dist[B] = min(10, 3+4) = 7
  Relax C->D: dist[D] = min(20, 3+11) = 14
  dist = {A:0, B:7, C:3, D:14, E:inf}
Iteration 3 - Select B (dist 7):
  Relax B->D: dist[D] = min(14, 7+5) = 12
  dist = {A:0, B:7, C:3, D:12, E:inf}
Iteration 4 - Select D (dist 12):
  Relax D->E: dist[E] = min(inf, 12+1) = 13
  dist = {A:0, B:7, C:3, D:12, E:13}
Iteration 5 - Select E (dist 13): no further updates.
Final distances from A:
  dist[A] = 0 (path: A)
  dist[B] = 7 (path: A -> C -> B)
  dist[C] = 3 (path: A -> C)
  dist[D] = 12 (path: A -> C -> B -> D)
  dist[E] = 13 (path: A -> C -> B -> D -> E)
```

📍 **المرجع:** Lecture 8 — Dijkstra example slides | likely: high

Step-by-step | Calculation | Shortest Path | Dijkstra

C4 **4. Given a weighted directed graph with n = 6 vertices and m = 12 edges, compute the total running time of (a) Kruskal's algorithm and (b) Dijkstra's algorithm with a binary heap. Show the dominant term.**

الترجمة: بالنظر إلى رسم بياني موجّه موزون بـ n = 6 رؤوس و m = 12 حافة، احسب الزمن الكلي لـ (a) خوارزمية Kruskal و (b) خوارزمية Dijkstra باستخدام binary heap. اعرض الحد المهيمن.

✅ **الحل النموذجي:**

```
Given: n = |V| = 6, m = |E| = 12.
(a) Kruskal's algorithm:
  - Sort edges: O(m log m) = 12 * log(12) = 12 * 3.585 = 43.02 (approximately,
in base-2).
  - MakeSet: O(n) = 6 operations.
  - FindSet: O(m) = 12 calls (two per edge).
  - Union: O(n) = up to 5 calls (n - 1).
  - Dominant term: O(m log m).
  - Using log2(12) approx 3.585, total work approx 12 * 3.585 = ~43 edge-
comparisons (plus union-find overhead).
(b) Dijkstra's algorithm with binary heap:
  - O((n + m) log n) = (6 + 12) * log2(6) = 18 * 2.585 = ~46.53.
  - Equivalently O(E log V) = 12 * 2.585 = ~31, but the (V + E) form also
accounts for the n DeleteMin operations.
Summary: Kruskal dominant term = O(m log m); Dijkstra dominant term = O((n + m)
log n) = O(E log V).
```

📍 **المرجع:** Lecture 8 — Running Time slides for both algorithms | likely: high

Calculation | Time Complexity | Dijkstra | Kruskal

C5 **5. Construct the weighted adjacency matrix AND the weighted adjacency list for the following undirected weighted graph. Vertices = {1, 2, 3, 4}. Edges (undirected, weighted): (1,2,5), (1,4,2), (2,3,8), (3,4,3), (2,4,1). Use infinity for non-existent edges and 0 on the diagonal.**

الترجمة: أنشئ مصفوفة التجاور الموزونة وقائمة التجاور الموزونة للرسم البياني غير الموجه الموزون التالي. الرؤوس = {1, 2, 3, 4}. الحافات: (1,2,5), (1,4,2), (2,3,8), (3,4,3), (2,4,1). استخدم infinity للحافات غير الموجودة و 0 على القطر.

✅ **الحل النموذجي:**

```
Vertices: 1, 2, 3, 4. Edges (undirected, so symmetric): (1,2,5), (1,4,2),
(2,3,8), (2,4,1), (3,4,3).
Weighted Adjacency Matrix A (size 4x4):
         1     2     3     4
    1 [  0     5   inf     2  ]
    2 [  5     0     8     1  ]
    3 [ inf    8     0     3  ]
    4 [  2     1     3     0  ]
Explanation:
  A[1][1] = A[2][2] = A[3][3] = A[4][4] = 0 (diagonal).
  A[1][2] = A[2][1] = 5 (edge 1-2).
  A[1][4] = A[4][1] = 2 (edge 1-4).
  A[2][3] = A[3][2] = 8 (edge 2-3).
  A[2][4] = A[4][2] = 1 (edge 2-4).
  A[3][4] = A[4][3] = 3 (edge 3-4).
  All other off-diagonal entries = infinity (no edge).
Weighted Adjacency List Adj:
  Adj[1] = [(2, 5), (4, 2)]
  Adj[2] = [(1, 5), (3, 8), (4, 1)]
  Adj[3] = [(2, 8), (4, 3)]
  Adj[4] = [(1, 2), (2, 1), (3, 3)]
Each list entry is a (neighbor, weight) pair.
```

📍 **المرجع:** Lecture 8 — Weighted Adjacency Matrix + (Weighted) Adjacency List Structure slides | likely: high

Calculation | Graph Representation | Adjacency List | Adjacency Matrix

C6 **6. For a directed graph G with V = {A, B, C, D} and E = {(A->B), (A->C), (B->D), (C->D), (D->A)}, compute indeg(v) and outdeg(v) for every vertex, and verify that sum of in-degrees equals sum of out-degrees equals |E|.**

الترجمة: بالنسبة للرسم البياني الموجه G بالرؤوس V = {A, B, C, D} والحافات E = {(A->B), (A->C), (B->D), (C->D), (D->A)}، احسب indeg(v) و outdeg(v) لكل رأس، وتحقق أن مجموع in-degrees يساوي مجموع out-degrees يساوي |E|.

✅ **الحل النموذجي:**

```
Edges (directed): A->B, A->C, B->D, C->D, D->A. So |E| = 5.
Out-degrees (edges leaving each vertex):
  outdeg(A) = 2 (A->B, A->C)
  outdeg(B) = 1 (B->D)
  outdeg(C) = 1 (C->D)
  outdeg(D) = 1 (D->A)
  Sum of out-degrees = 2 + 1 + 1 + 1 = 5.
In-degrees (edges entering each vertex):
  indeg(A) = 1 (from D->A)
  indeg(B) = 1 (from A->B)
  indeg(C) = 1 (from A->C)
  indeg(D) = 2 (from B->D and C->D)
  Sum of in-degrees = 1 + 1 + 1 + 2 = 5.
Verification:
  sum of out-degrees = 5 = |E|. (matches)
  sum of in-degrees = 5 = |E|. (matches)
The directed form of the Hand-Shaking Lemma is verified.
Also, deg(v) = outdeg(v) + indeg(v):
  deg(A) = 3, deg(B) = 2, deg(C) = 2, deg(D) = 3.
```

📍 **المرجع:** Lecture 8 — Hand-Shaking Lemma (directed form) | likely: high

Verification | Calculation | Directed Graph | Hand-Shaking Lemma